27

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Apache Kafka İç Mimarisi: Topics, Partitions, Consumer Groups ve Segmentler

Apache Kafka Internals: Topics, Partitions, Consumer Groups & Segments

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram

```
[ KAFKA TOPIC: "orders" (3 PARTITION'A BÖLÜNMÜŞ) ]
Partition 0: [0][1][2][3][4] ──► Consumer 1 (Pod A)
Partition 1: [0][1][2][3]    ──► Consumer 2 (Pod B)   Consumer Group: "Billing-Service"
Partition 2: [0][1][2]       ──► Consumer 3 (Pod C)
(Her partition yalnızca TEK bir consumer tarafından okunur -> Paralellik & Sıralılık!)

[ PARTITION DISK YAPISI: .log ve .index DOSYALARI ]
Segment 000000.log  (1 GB Dolunca kapanır) ──► Segment 000001.log (Yeni aktif kütük)
Segment 000000.index (Seyrek İndeks - Sparse Index: Binary Search ile O(log N) arama)
```

💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kafka Topic'ini 3 şeritli bir otoyol (3 Partition) olarak düşünün. Eğer tek şerit olsaydı tüm arabalar tek sıra halinde giderdi ve trafik tıkanırdı. 3 şerit açarak trafiği 3 katına çıkarırsınız! Şeritlerin çıkışına da 3 kişilik bir gişe görevlisi ekibi (Consumer Group) koyarsınız: Her görevli yalnızca KENDİ şeridini izler. Arabalar kendi şeritlerinde asla birbirini sollayamaz (Şerit içinde kesin sıra garantisi!); ancak şeritler birbirini beklemez!

🇬🇧 ENGLISH (ELI5)

Think of a Kafka Topic as a 3-lane highway (3 Partitions). A single-lane road bottlenecks all cars into a single file crawl. Adding 3 lanes triples traffic throughput! At the highway exit, assign 3 toll clerks belonging to the same union (Consumer Group): each clerk exclusively monitors their dedicated lane. Within a single lane, cars cannot overtake one another (strict in-order sequence within a partition!); but different lanes progress in parallel!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kafka mimarisinin temel yapı taşları: 1) Topic: Mantıksal veri akışı kategorisi. 2) Partition: Yatay ölçekleme ve paralellik birimidir. Mesajlar anahtarın (Key) hash değerine göre (`MurmurHash2(key) % partition_count`) ilgili bölüme yazılır. Aynı anahtara sahip tüm mesajlar KESİNLİKLE aynı partition'a gider ve sıralı işlenir. 3) Consumer Group: Bir grup içindeki tüketici sayısı partition sayısını aşamaz; her partition o gruptaki yalnızca TEK bir tüketiciye atanır. 4) Disk Segmentleri: Her partition diskte 1 GB'lık segment dosyalarına bölünür (`.log` ve seyrek indeks `.index`). İşletim sistemi sayfa önbelleği (PageCache) kullanılarak disk aramaları RAM hızına çıkarılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Kafka internal architecture: 1) Partitioning: The fundamental unit of scalability. Keyed messages route via `MurmurHash2(key) % num_partitions`, guaranteeing deterministic partition assignment and strict ordering for identical keys (e.g. all events for `user_101`). 2) Consumer Groups: Workload parallelization engine. A partition is consumed by at most one worker per consumer group; exceeding worker counts relative to partition counts leaves surplus consumers idle. 3) Storage Segments: Partitions split on disk into bounded segment files (typically 1GB `.log` alongside `.index` sparse indexes). Kafka relies heavily on the Linux OS PageCache rather than JVM heap memory, avoiding garbage collection pauses while maximizing sequential disk IO.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Kafka Partition Yönlendirme ve Consumer Group Eşleme Simülasyonu
import java.util.*;

public class KafkaPartitionRouter {
    private final int partitionCount;

    public KafkaPartitionRouter(int partitionCount) {
        this.partitionCount = partitionCount;
    }

    public int assignPartition(String messageKey) {
        if (messageKey == null) {
            return ThreadLocalRandom.current().nextInt(partitionCount); // Round-Robin
        }
        // Aynı anahtar DAİMA aynı partition'a gider!
        return Math.abs(messageKey.hashCode() % partitionCount);
    }
}

void main() {
    var router = new KafkaPartitionRouter(3); // 3 Partition'lı Topic
    IO.println("User 'A' Olayı -> Partition #" + router.assignPartition("user:A"));
    IO.println("User 'B' Olayı -> Partition #" + router.assignPartition("user:B"));
    IO.println("User 'A' İkinci Olayı -> Partition #" + router.assignPartition("user:A") + " (Sıra
Korundu!)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Tüm sistem için global bir mesaj sırası garanti ederim' derseniz elenirsiniz! Kafka'da mesaj sırası GLOBAL DEĞİLDİR; sadece AYNI PARTITION içinde sıralıdır. Global sıra istiyorsanız Topic'i 1 Partition'a düşürmelisiniz; bu da yatay ölçeklemeyi yok eder.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never claim Kafka guarantees global ordering across an entire topic. Kafka strictly guarantees ordering ONLY WITHIN A SINGLE PARTITION. Global ordering requires restricting a topic to 1 single partition, which demolishes horizontal throughput.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Kafka'da paralellik sınırı partition sayısı kadardır.
- Aynı varlığa ait mesajlar aynı Key ile gönderilerek sıralı işleme garantilenir.
- Kafka JVM heap yerine işletim sistemi PageCache'ini kullanarak GC duraklamalarını sıfırlar.

🎯 Key Takeaways for the Interview

- The partition count establishes the hard ceiling on consumer parallelism per consumer group.
- Passing consistent message keys guarantees partition-level ordering for state machines.
- Kafka relies on OS PageCache rather than JVM heap, neutralizing GC pauses entirely.